Integrative Programming

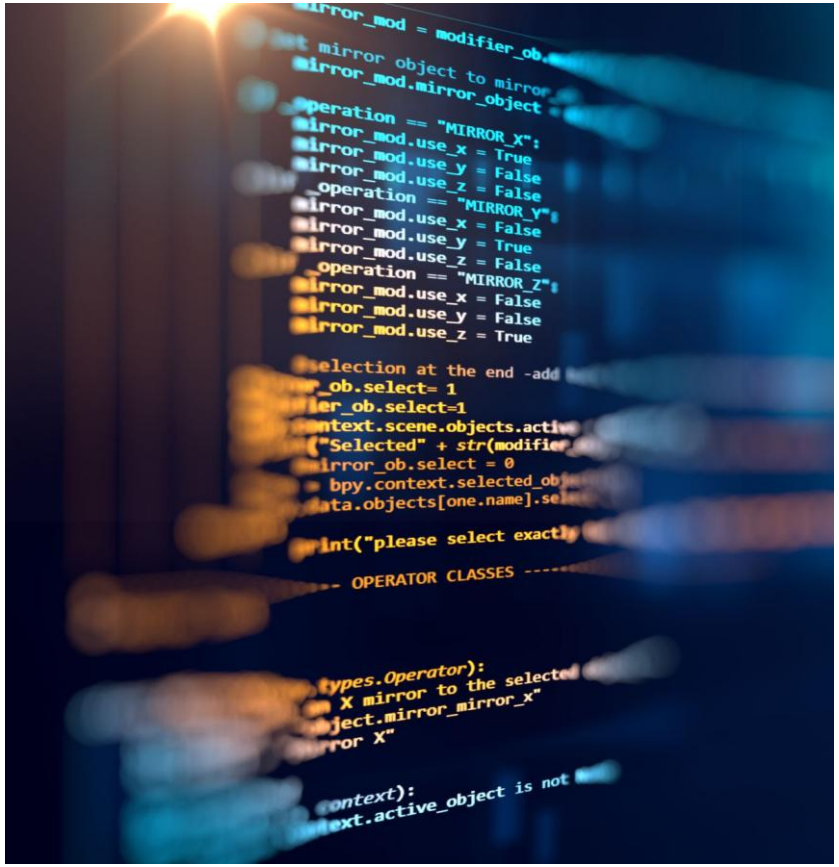
Introduction and Web Foundations

Overview

Integrative Programming focuses on building applications that combine multiple technologies and languages.

By the end of this lecture, students should understand:

- What integrative programming is
- How the web works at a high level
- The basic tools needed to start development
- Core HTML5 and CSS3 concepts



What is Integrative Programming?

Integrative Programming is the practice of combining:

- Multiple programming languages (e.g., PHP, JavaScript, SQL)
- Different technologies (frontend, backend, databases)
- Various systems and services into one working application

Deals with an integration approaches and techniques to enable a secure intra and inter application collaboration that connect different components of IT infrastructure such as:

- People
- Applications
- Platforms and
- Databases

Goal: Build **complete, functional, real-world systems**

Why Integrative Programming Matters?

- Modern applications are not built with one language only
- Web apps require coordination between:
  - User interface
  - Server-side logic
  - Data storage
- Industry demand for full-stack and adaptable developers
- Examples:
  - E-commerce websites
  - Learning management systems
  - Online booking systems

Multi-Language Web Applications

Typical web stack:

- **Frontend:** HTML, CSS, JavaScript
- **Backend:** PHP
- **Database:** MySQL

- Each layer has a specific role but must work together seamlessly
- Example: **Facebook** The social media giant was originally and is still primarily built on PHP, having developed custom tools like the HipHop Virtual Machine (HHVM) to optimize its performance and handle billions of active users.

Web Architecture Overview

- **Web Architecture** defines how components of a web system interact
- Key Concept: **Client-Server Model**
- **Client:** Web browser (Chrome, Firefox, Edge)
- **Server:** Hosts the application and processes requests
- Flow:

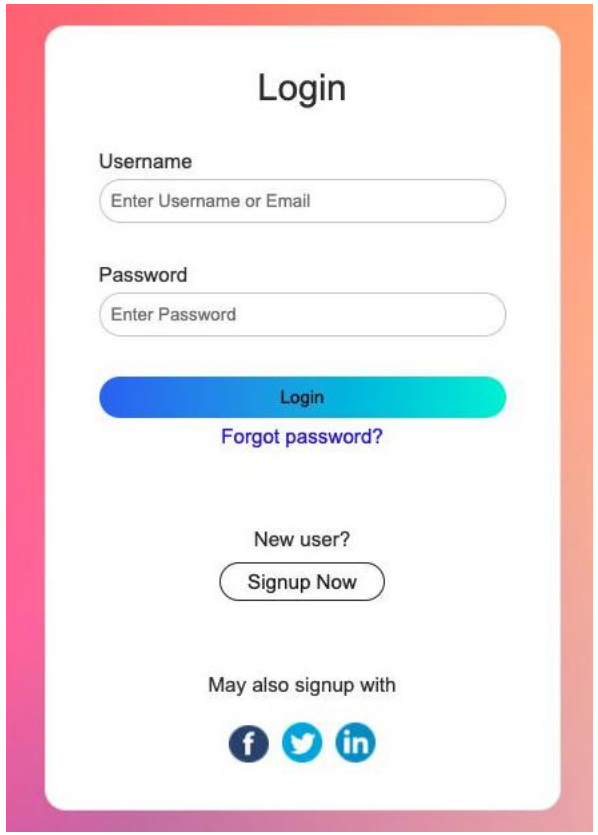
Web Foundations - Protocols

HyperText Transfer Protocol (HTTP) - The communication protocol used on the web

- It defines how requests and responses are sent

Common HTTP methods:

- **GET** – retrieve data
- **POST** – send data
- It is commonly used in submitting a login form.

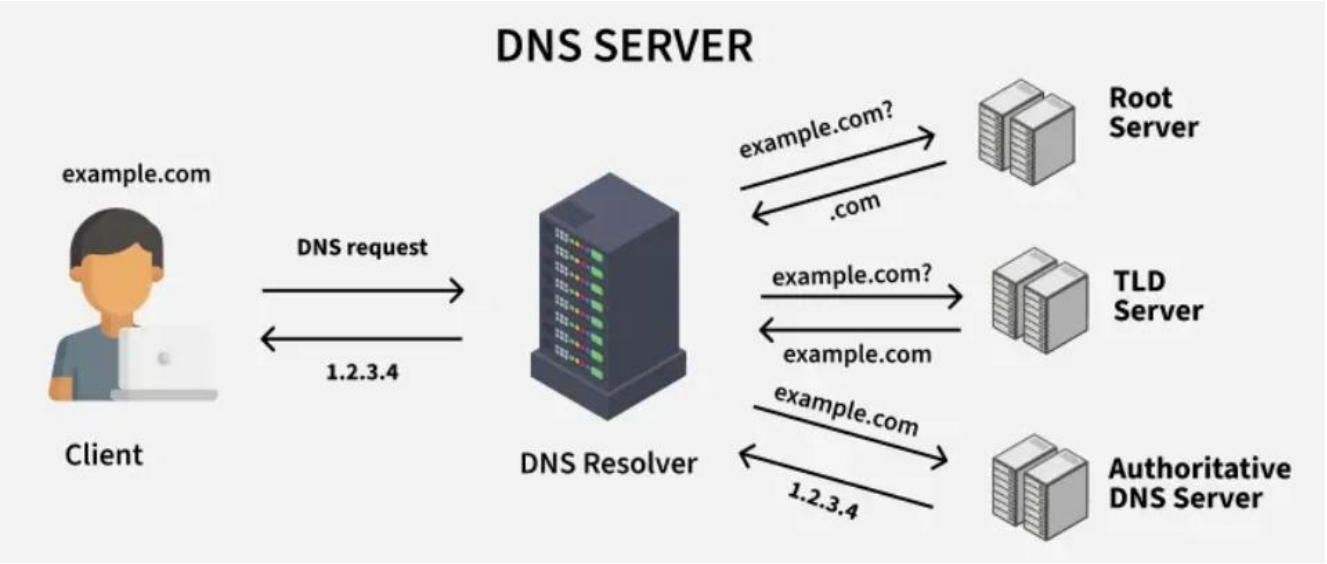


Web Foundations - Protocols

Domain Name System (DNS) – It translates domain names into IP Addresses. Humans remember names, computers use numbers

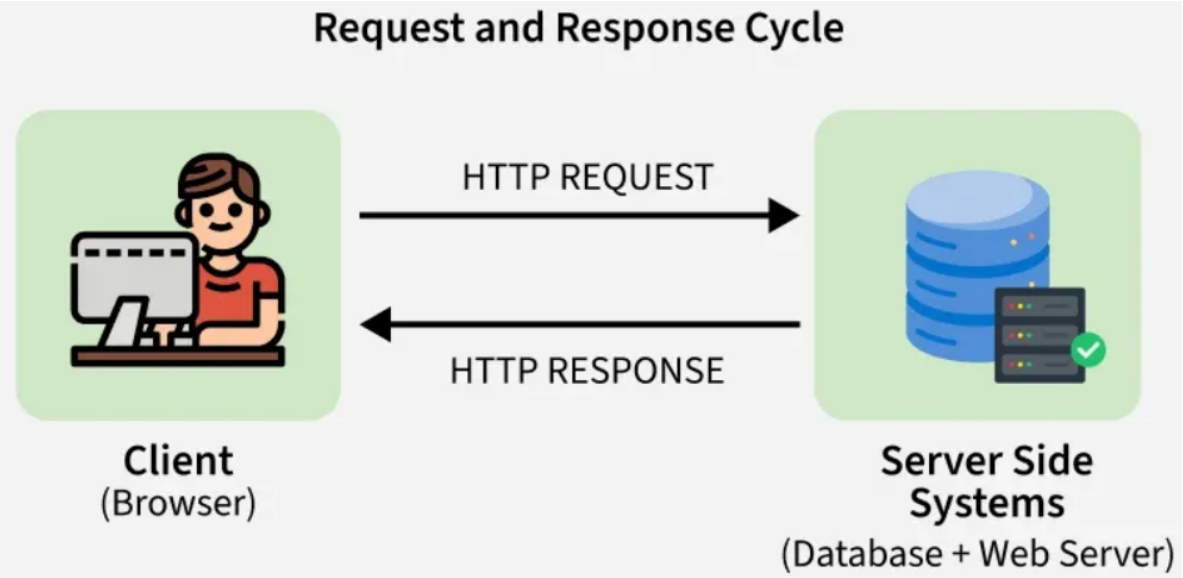
Example:

www.google.com 142.250.xxx.xxx



Web Request Life Cycle

- It describes the sequence of steps that occur when a client (e.g. web browser) requests a resource from a web server and receives a response.
- Also known as the **HTTP Request**.
- How it works:
  1. User enters a URL
  2. DNS resolves the domain name
  3. Browser sends HTTP request
  4. Server processes request
  5. Response is returned and displayed



Types of HTTP Requests

- **GET:** Retrieves data from the server without modifying any data.
- **POST:** Sends data to the server to create or update resources.
- **PUT:** Updates an existing resource or creates it if absent.
- **DELETE:** Requests the deletion of a resource on the server.
- **PATCH:** Makes partial updates to an existing resource.
- **HEAD:** Retrieves only the headers of a resource, not the body.
- **OPTIONS:** Queries the server for supported HTTP methods and features.

Development Environment Overview

To build web application locally, we need:

Web Server – a is software that receives requests from a browser and sends back web pages or data. It delivers HTML pages and runs server-side scripts (like PHP) before sending results to the client.

- Example: Apache (used in XAMPP/WAMP/LAMP)

Database Server - stores, organizes, and manages application data. It allows programs to save, retrieve, update, and delete information efficiently.

- Example: MySQL or MariaDB

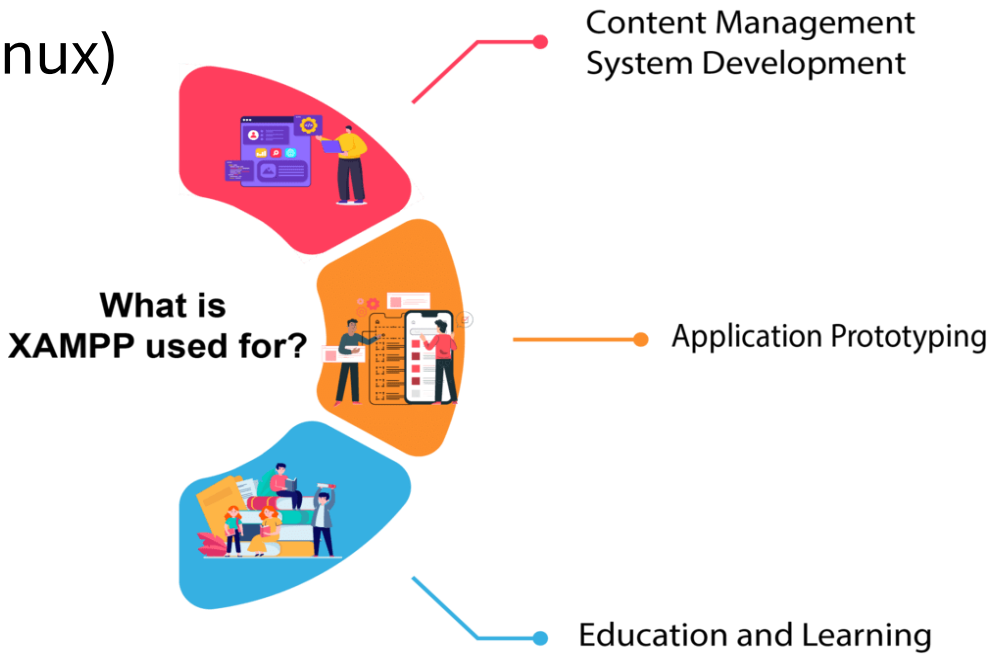
PHP Interpreter - is the program that reads and executes PHP code on the server. It processes PHP scripts and converts them into HTML that the browser can display.

- Example: PHP engine included in XAMPP/WAMP/LAMP

These three components work together to allow developers to build, test, and run dynamic web applications locally.

XAMPP / WAMP / LAMP

- **XAMPP:** Cross-platform (Windows, macOS, Linux)
- **WAMP:** Windows
- **LAMP:** Linux
- Components
  - **Apache** (Server)
  - **MySQL** (Database)
  - **PHP and Perl** (Interpreter)



X	A	M	P	P
Cross-Platform	Apache	MySQL	PHP	Perl

Why use a local server?

Test Applications Without Internet

A local server allows developers to run and test web applications entirely on their own computer.

This means developers can continue coding and testing even without an internet connection, which is especially useful in labs or at home.

Safe Environment for Learning

Using a local server provides a controlled and private environment.

Mistakes will not affect real users or live websites, allowing students to experiment freely, learn from errors, and reset their projects without risk.

Why use a local server?

Faster Development and Debugging

Because everything runs locally, applications load much faster compared to online servers.

This speed makes it easier to test changes, identify errors, and debug code efficiently during development.

📌 All PHP Code Must Run on a Server

PHP is a server-side language, meaning it cannot run directly in a browser.

A local server (such as Apache in XAMPP) is required to process PHP scripts and send the resulting HTML to the browser.

HTML5 Refresher

HTML (Hypertext Markup Language) Structures Web Content

- **HTML** is the foundation of all web pages.
- It defines the structure and meaning of content such as headings, paragraphs, images, links, and forms.
- HTML does not control design or behavior—its main role is to organize information so browsers and other technologies can understand it.

Semantic Element in HTML5

HTML5 introduces semantic elements, which describe the purpose of content instead of just how it looks.

Examples:

- `<header>` – Contains introductory content or navigation links
- `<nav>` – Holds navigation menus
- `<section>` – Groups related content together
- `<footer>` – Contains page or section footer information

Using semantic elements improves:

- Code readability
- Accessibility (for screen readers)
- Search engine optimization (SEO)

Better Form Controls

HTML5 improves user input by adding new and enhanced form elements.

These controls help validate data automatically and improve user experience without extra scripting.

Examples include:

- Email input fields
- Date pickers
- Required field validation
- This reduces errors and makes forms easier to use.

Multimedia Support

HTML5 allows **audio and video content** to be embedded directly into web pages **without third-party plugins**.
This makes multimedia more accessible, faster, and compatible across devices.

Common use cases:

- Video tutorials
- Background audio
- Embedded lectures or demonstrations

Basic HTML Structure

```
<!DOCTYPE html>
<html>
<head>
  <title>My First Page</title>
</head>
<body>
  <h1>Hello World</h1>
</body>
</html>
```

CSS3 Refresher

CSS (Cascading Style Sheets) Controls Presentation

- CSS is responsible for the **visual appearance** of a web page.
  It controls how HTML elements look, including layout, colors, fonts, spacing, and alignment.
- By separating design from structure, CSS makes web pages easier to maintain and update.

Flexible Layouts (Flexbox)

- Flexbox is a CSS3 layout system designed to arrange elements **efficiently within a container**.
- It allows items to automatically adjust their size and position, making layouts easier to build and manage.
- Flexbox is commonly used for:
  - Navigation bars
  - Page sections
  - Aligning content vertically and horizontally
  - It reduces the need for complex positioning and floating elements.

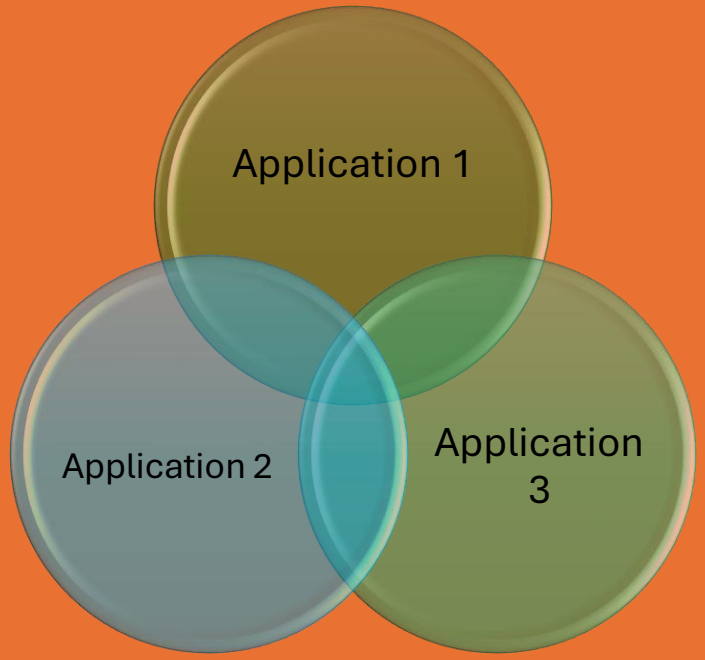
Responsive Design

- Responsive design ensures that web pages **adapt to different screen sizes**, such as desktops, tablets, and mobile phones.
- CSS3 provides tools like flexible units, media queries, and flexible layouts to achieve responsiveness.
- This is important because users access websites using many different devices.

Transitions and Animations

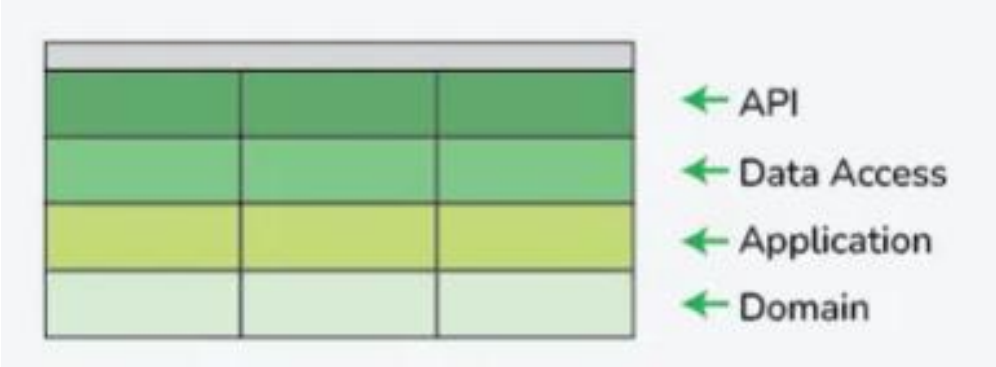
- CSS3 allows developers to create smooth visual effects without JavaScript.
- Transitions control how changes happen over time (e.g., hover effects), while animations allow more complex movement.
- These features enhance:
  - User experience
  - Visual feedback
  - Interactivity

Separation of Concerns



Separation of Concerns

- **Separation of Concerns** is a design principle that encourages dividing a program into distinct sections, where each section handles a specific responsibility.
- In web development, this means separating **structure**, **design**, and **logic** into different technologies.
- This approach makes applications easier to understand, develop, and maintain.



HTML → Structure

- HTML is responsible for the **content and structure** of a web page.
- It defines elements such as headings, paragraphs, forms, and sections, focusing on *what* content exists rather than *how* it looks.
- Using HTML only for structure ensures that content remains accessible and well-organized.

CSS → Design

- CSS controls the **visual presentation** of the web page.
- It handles layout, colors, fonts, spacing, and responsiveness without changing the underlying content.
- By keeping design in CSS, developers can update the appearance of a website without modifying the HTML structure.

PHP → Logic

- PHP handles the **application logic** on the server side.
- It processes data, performs calculations, interacts with databases, and decides what content should be displayed.
- Separating logic into PHP prevents complex operations from cluttering HTML and CSS files.

📌 Clean, maintainable code depends on separation

How everything fits together?

1. Browser Requests a Page

- When a user types a website address or clicks a link, the **browser sends a request** to the server.
  This request asks for a specific page or resource, such as an HTML or PHP file.

2. Server Runs PHP Code

- If the requested page contains PHP, the **server processes the PHP code first**.
  The browser never sees the PHP code—it is executed entirely on the server.
- PHP may:
  - Process user input
  - Retrieve data from a database
  - Make decisions based on logic

How everything fits together?

3. PHP Outputs HTML

- After executing, PHP **generates HTML output**.
  This HTML represents the final content that will be sent back to the browser.
- At this point, PHP has finished its job.

4. CSS Styles the Page

- Once the browser receives the HTML, it applies the **CSS rules** linked to the page.
- CSS controls how the content looks, including layout, colors, fonts, and responsiveness.

This Integration Is the Core of Integrative Programming

Integrative Programming

Introduction and Web Foundations